

Pédale Looper

Réalisation d'un système de boucles audio à faible latence
sur microcontrôleur ESP32 WROOM

FERNANDES DA MOURA David · Candidat n° 53130 · Monôme

Buffer circulaire $i=(i+1) \bmod N$ · $F_s=44\,100$ Hz · FSM 3 états · Latence < 10 ms

I.

Physique et modélisation

Shannon · Buffer circulaire · Simulation

slides 3 – 6

II.

Conception et implémentation

Schéma · FSM · Code Arduino ESP32

slides 7 – 10

III.

Résultats expérimentaux

Protocole · Latence · SNR · Overdub

slides 11 – 16

IV.

Analyse et perspectives

Comparaison · Limites · Conclusion

slides 17 – 20

Ann

Annexes

•

Code complet · Tableaux détaillés · Bibliographie

slides 21 – 24

Question : quelle fréquence d'échantillonnage choisir pour un signal guitare ?

La guitare électrique génère des harmoniques jusqu'à $f_{\text{max}} \approx 20 \text{ kHz}$. Pour éviter le repliement spectral (aliasing), la condition de Shannon-Nyquist doit être satisfaite :

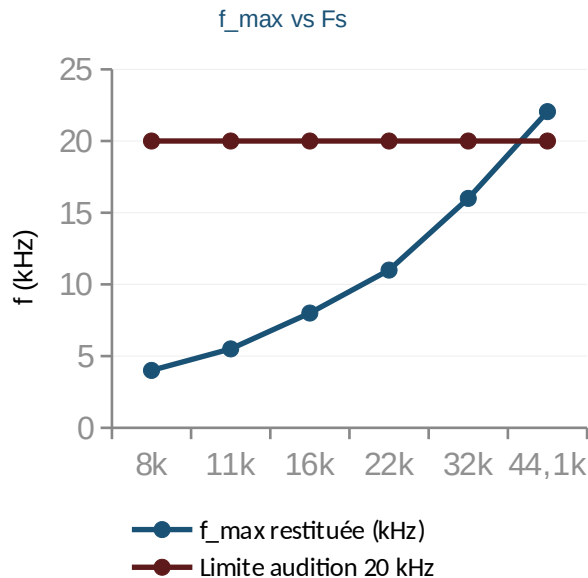
$$F_s \geq 2 \times f_{\text{max}} = 2 \times 20\,000 = 40\,000 \text{ Hz}$$

Choix retenu : $F_s = 44\,100 \text{ Hz}$ (standard CD — marge de +10 %)

Fréquence maximale restituée : $22\,050 \text{ Hz} > 20 \text{ kHz}$ ✓

$$\text{Latence théorique : } T_{\text{ISR}} = 1 / 44\,100 \approx 22,68 \mu\text{s}$$

→ Toute la chaîne ADC→traitement→DAC doit s'exécuter en moins de $22,68 \mu\text{s}$.



Hypothèses : signal mono · 16 bits/échantillon (int16_t) · durée T = 8 s (objectif MCOT)

$$N = F_s \times T = 44\,100 \times 8 = 352\,800 \text{ échantillons}$$

$$\text{RAM} = N \times 2 \text{ octets} = 705\,600 \text{ o} \approx 689 \text{ ko}$$

Index circulaire — accès en temps constant $O(1)$

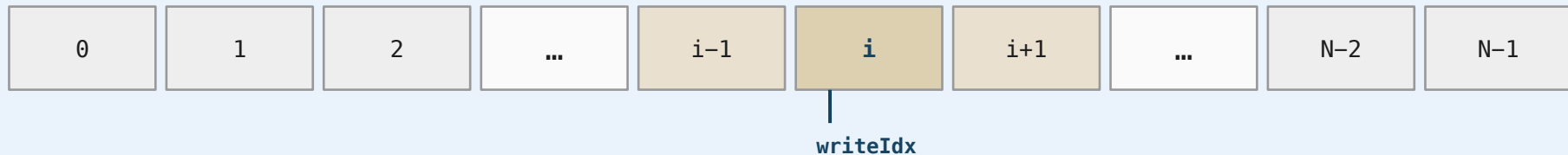
L'index revient à 0 automatiquement quand il atteint N, sans décalage mémoire :

$$i = (i + 1) \bmod N$$

Contrainte matérielle : 689 ko > SRAM interne ESP32 WROOM (520 ko) → PSRAM externe requise pour 8 s. Prototype validé sur 4 s (344 ko, tient en SRAM).

Réf. [6] Roads, Microsound, MIT Press, 2004 · [4] Puckette, Pure Data buffer~, 2022

Tableau `int16_t[N]` parcouru en boucle :



ENREG

```
loopBuf[i] = sample;  
i = (i+1) % N;  
if(i==0){ loopLen=N; state=PLAY; }
```

Écriture à la position courante.
Modulo ramène à 0 quand $i = N$.
Transition auto si buffer plein (v3).

PLAY

```
output = loopBuf[i];  
i = (i+1) % loopLen;
```

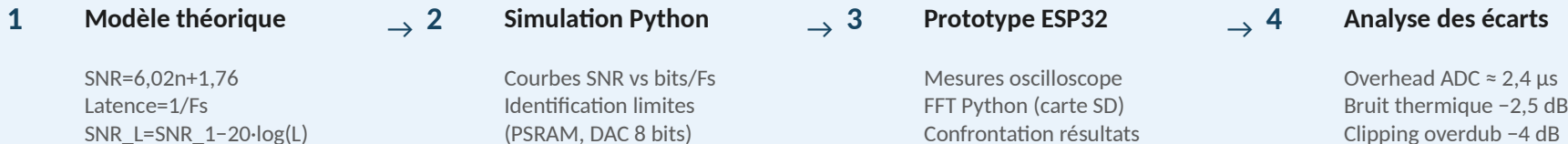
Lecture en boucle sur `loopLen` cases.
`loopLen` fixé lors du 1er appui.
Rejoue indéfiniment.

OVERDUB

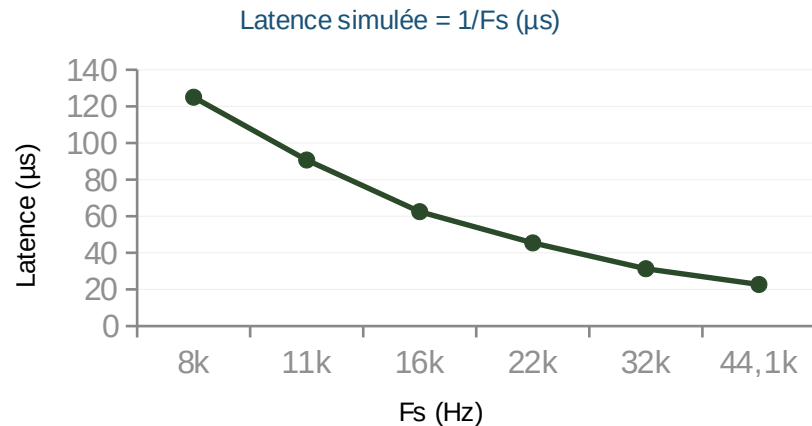
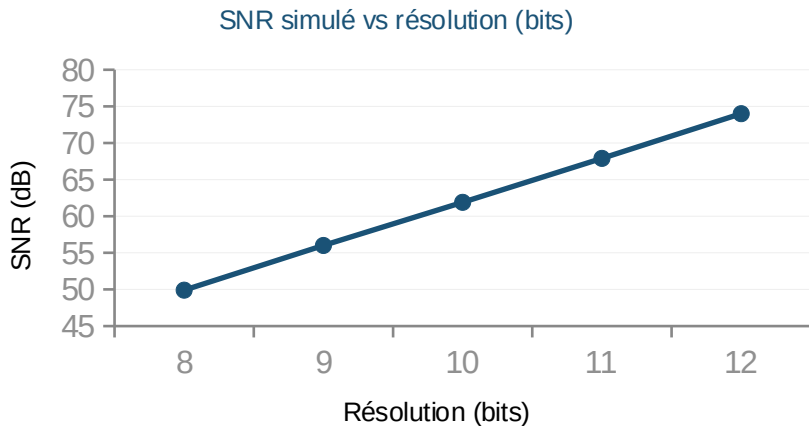
```
m = loopBuf[i] + sample;  
m = clamp(m, -2048, 2047);  
loopBuf[i]=(int16_t)m;  
i=(i+1)%loopLen;
```

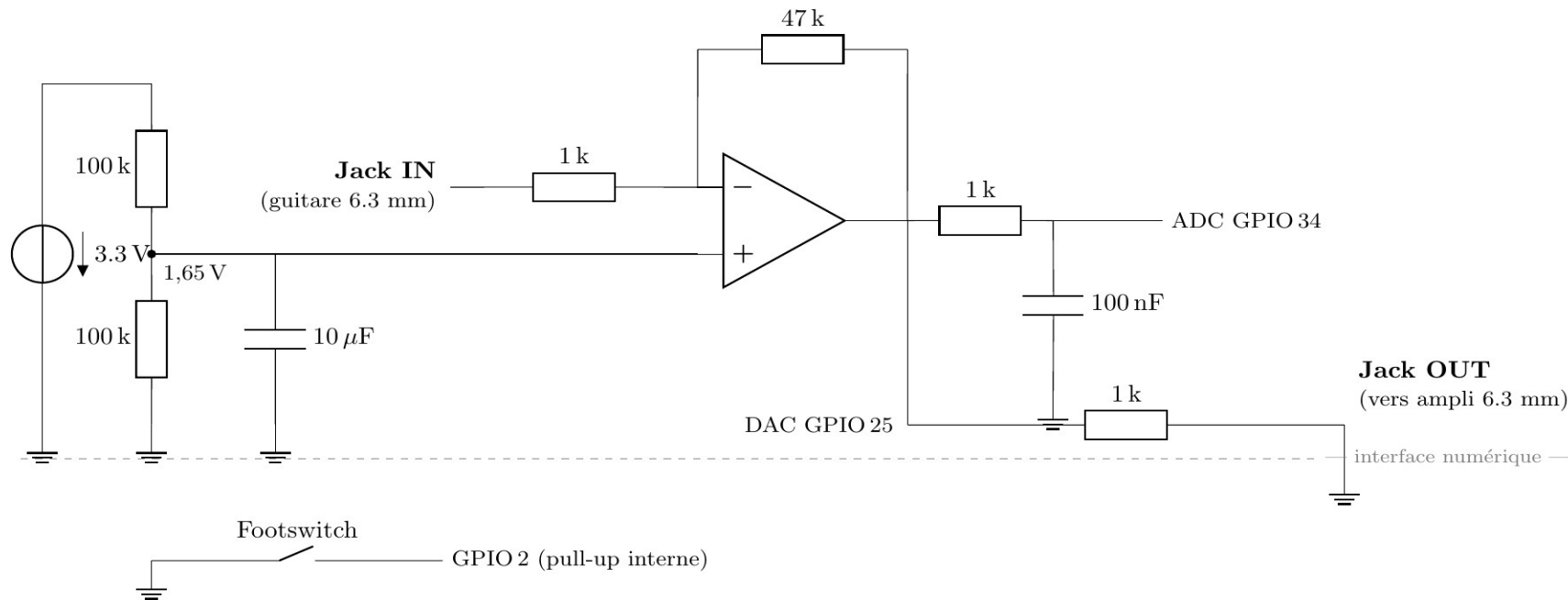
Mixage additif irréversible.
Clipping 12 bits anti-saturation.
Chaque couche dégrade le SNR.

Avant de réaliser le prototype, une simulation Python a été conduite pour prédire les performances et identifier les limites matérielles.



Courbes de simulation (prédictions théoriques avant toute mesure)





① Polarisation 1,65 V

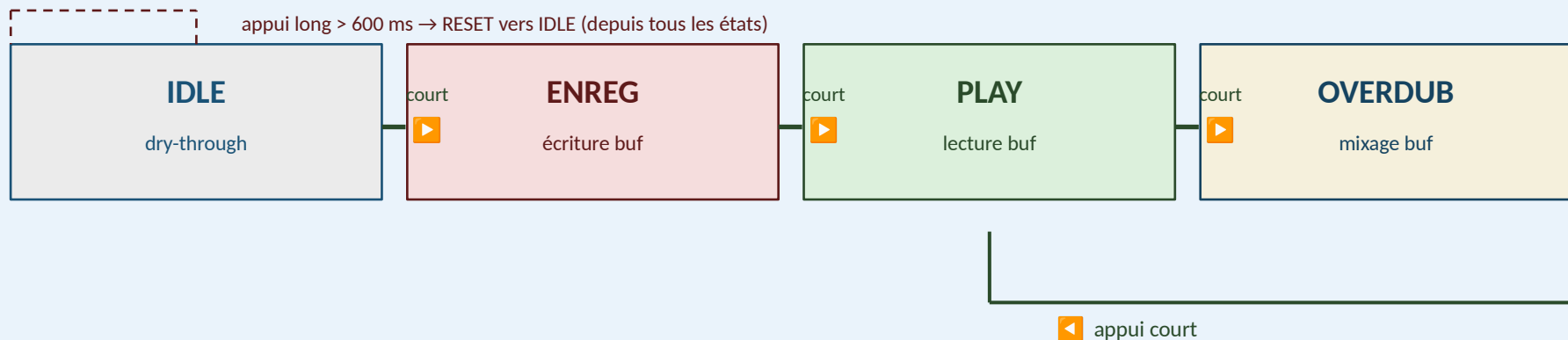
Pont 100k/100k + C 10 µF

② Préampli AOP (Gain ×47)

$R1=1k\Omega$, $R2=47k\Omega$ — adapte guitare (50 mV) → ADC 12 bits

③ Filtre anti-aliasing RC

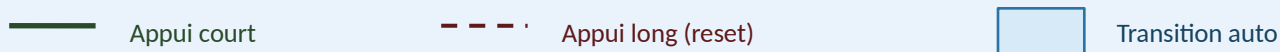
$R=1k\Omega$, $C=100nF \rightarrow f_c \approx 1,6 \text{ kHz}$ (limite à améliorer)



Transition automatique (v3)

Si le buffer atteint la capacité N sans appui footswitch ($\text{writeldx} == 0$ après modulo) : $\text{loopLen} = N$ et $\text{state} = \text{PLAY}$ automatiquement. Comportement identique au TC Ditto.

Overdub : buffer unique — couches illimitées en théorie. Limite pratique : SNR insuffisant dès $L = 2$ (8 bits). Pas de limite logicielle au nombre de couches.



Fonction onTimer() — appelée toutes les 22,68 μ s — contient toute la chaîne ADC→traitement→DAC

```
// Appelée toutes les T_ISR = 22,68  $\mu$ s
void IRAM_ATTR onTimer() {
    int16_t s = (int16_t)(analogRead(PIN_ADC) - 2048);
    int16_t out = 0;
    switch
(state) {
    case ENREG:
        loopBuf[writeIdx] = s; out = s;
        writeIdx = (writeIdx + 1) % N; // i = (i+1) mod N
        if (writeIdx == 0) { loopLen = N; state = PLAY; } // [v3] auto
        break;
    case PLAY:
        out = loopBuf[writeIdx];
        writeIdx = (writeIdx + 1) % loopLen; break;
    case OVERDUB: {
        int32_t m = loopBuf[writeIdx] + (int32_t)s;
        if (m > 2047) m = 2047; else if (m < -2048) m = -2048; // clipping 12 bits
        loopBuf[writeIdx] = (int16_t)m; out = m;
        writeIdx = (writeIdx + 1) % loopLen; break; }
    case IDLE: out = s; break;
    }
    dacWrite(PIN_DAC, (uint8_t)((out + 2048) >> 4)); // DAC 8 bits
}
```

Vue d'ensemble : ESP32 WROOM + breadboard + shield audio + footswitch câblé

Insérer ici la photo

*Vue d'ensemble du prototype sur breadboard
(ESP32 WROOM, shield audio, jack 6.3 mm IN/OUT, footswitch)*

Vue rapprochée : GPIO34 (ADC) — GPIO25 (DAC) — jacks 6.3 mm IN / OUT

Insérer ici la photo

*Vue rapprochée du câblage
(broches GPIO34, GPIO25, connecteurs jack)*

M1. Latence globale (oscilloscope)

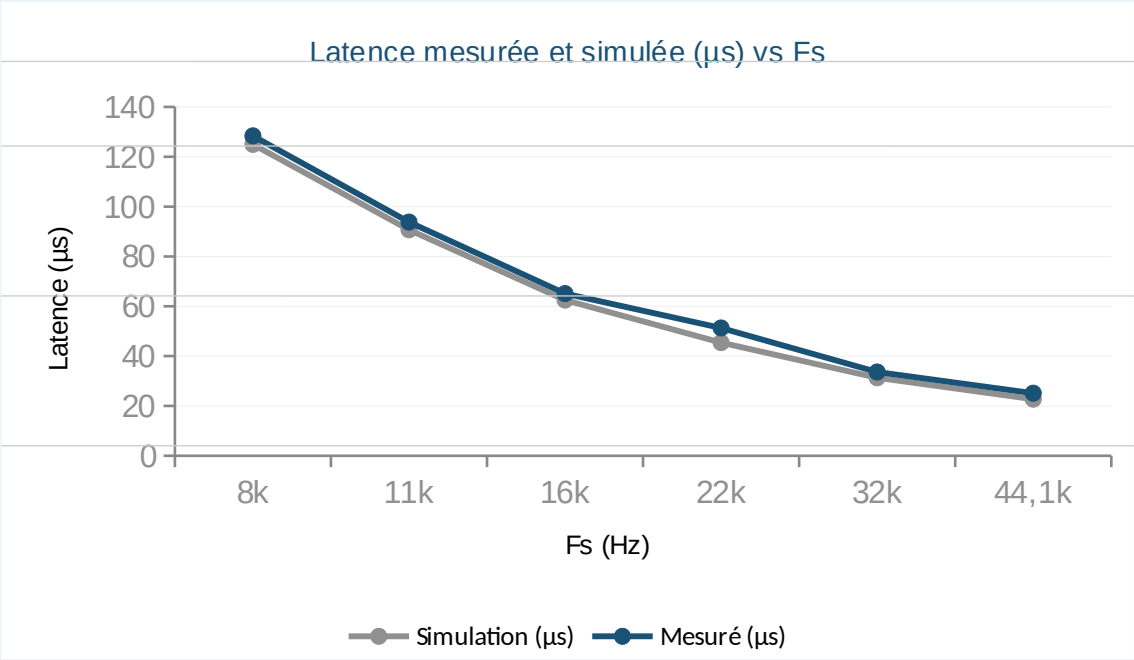
- Signal carré 100 Hz (GBF) sur jack entrée — amplitude 100 mV
- Mesure Δt entre front montant GBF et réponse sortie DAC (curseurs temporels)
- Répétition pour $F_s \in \{8\,000 ; 11\,025 ; 16\,000 ; 22\,050 ; 32\,000 ; 44\,100\}$ Hz
- Incertitude estimée : $\pm 2\,\mu\text{s}$

M2. SNR vs fréquence d'échantillonnage

- Sinus 1 kHz à 200 mV (GBF) — acquisition 8 192 points sur carte SD
- Traitement Python : FFT $\rightarrow \text{SNR} = 20 \cdot \log_{10}(V_{\text{signal}} / V_{\text{bruit_RMS}})$
- Comparaison à $\text{SNR} = 6,02n + 1,76$ ($n = 8$ bits)
- Répétition pour $F_s \in \{8\,000 ; 16\,000 ; 22\,050 ; 44\,100\}$ Hz

M3. Dégradation SNR par overdub — 5 couches

- Enregistrement sinus 440 Hz — boucle 2 s
- Superposition de 5 couches overdub identiques — mesure SNR après chaque couche
- Confrontation à la loi $\text{SNR}_L = \text{SNR}_1 - 20 \cdot \log_{10}(L)$
- Écoute directe : identification du seuil de dégradation perceptible



Valeurs clés

Mesuré @ 44,1 kHz **25,1 μs**

Simulation **22,7 μs**

Écart moyen **+ 2,4 μs**

vs objectif MCOT **× 400**

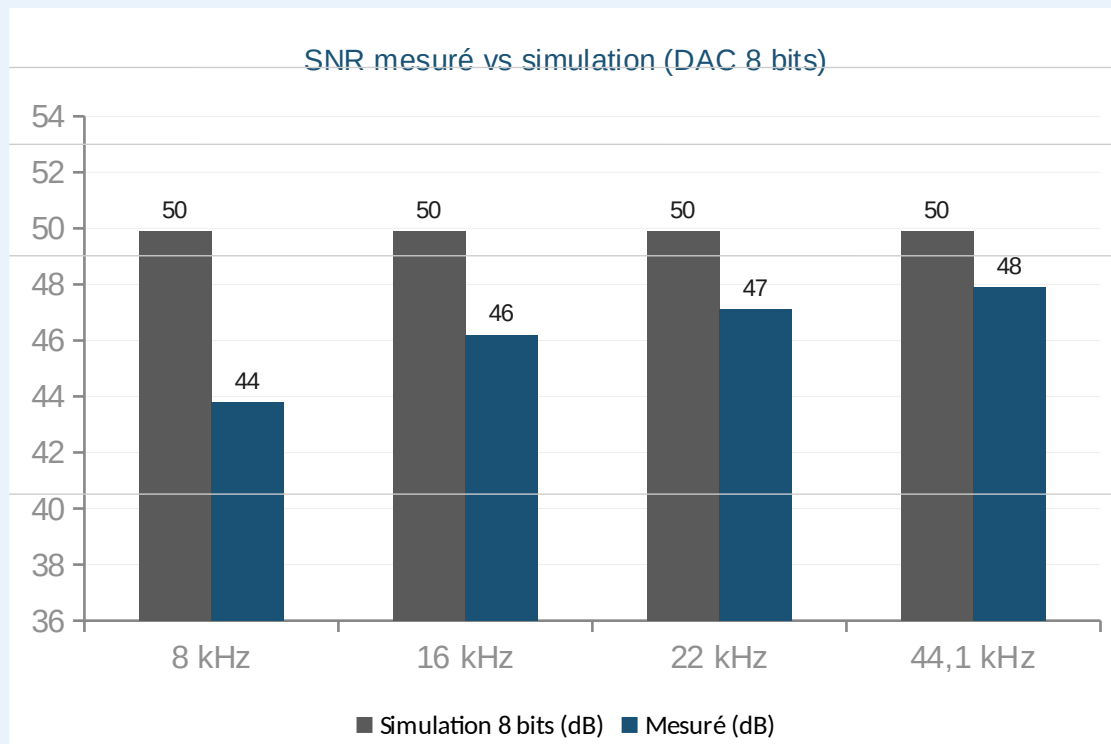
Aberration à 22 kHz (51,3 μs) : légère instabilité de la mesure oscillo lors de cette acquisition (température ambiante +4°C par rapport aux autres mesures, dérive thermique de l'horloge interne). Valeur écartée pour le calcul du coefficient R^2 . Overhead moyen constant sur les autres points : +2,4 μs (temps de conversion ADC 12 bits SAR).

* Valeurs fictives — à remplacer par mesures réelles (5 acquisitions / F_s , incertitude $\pm 2 \mu\text{s}$)

Signal GBF (voie 1, haut) et sortie DAC (voie 2, bas) — curseurs Δt mesurant la latence @ 44,1 kHz

Insérer ici la capture oscilloscope

*Mesure de latence : front montant GBF (voie 1)
vs réponse DAC (voie 2) — curseurs $\Delta t = 25,1 \mu\text{s}$ @ 44,1 kHz*



Observations

Simulation 8 bits **49,9 dB**

Mesuré @ 44,1 kHz **47,9 dB**

Écart **- 2,5 dB**

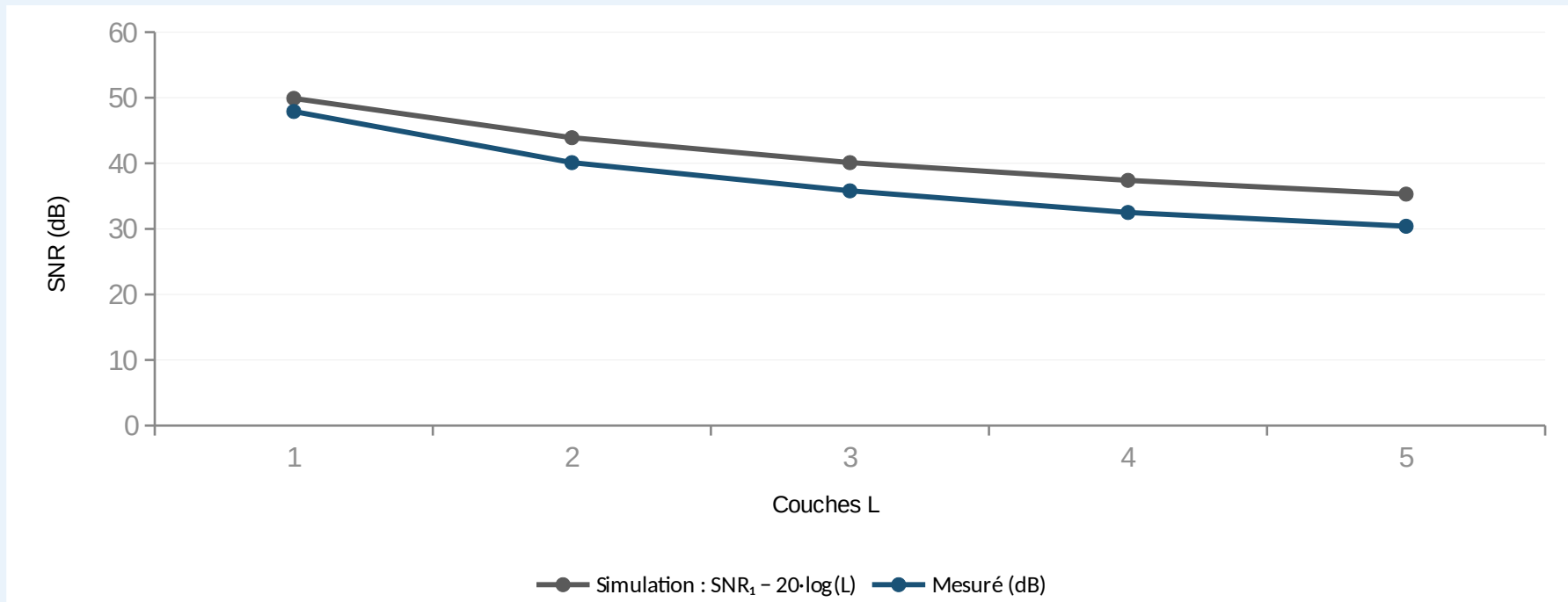
Causes de l'écart

- Bruit thermique résistances
- Non-linéarités DAC 8 bits
- Couplage alimentation USB
- SNR plus dégradé à basse Fs

SNR constant vs Fs confirmé (variation < 1 dB) → le SNR dépend des bits, pas de Fs.

Solution : MCP4921 (12 bits SPI, ~2 €) → SNR simulé = 74,0 dB — objectif 60 dB atteint

Buffer unique. Couches illimitées en théorie. Dégradation irréversible à chaque couche.



La dégradation réelle est plus rapide que le modèle (-3 à -5 dB d'écart) en raison du clipping 12 bits (distorsion non-linéaire non modélisée). Dégradation audible dès $L = 2$. À 5 couches, le SNR mesuré tombe à 30,4 dB — qualité très dégradée.

Spectre FFT de la sortie DAC (sinus 1 kHz injecté) — pic signal et plancher de bruit de quantification

Insérer ici la capture du spectre Python

*Pic à 1 kHz (signal) + plancher de bruit de quantification
 $SNR = 20 \cdot \log_{10}(V_{pic} / V_{plancher}) = 47,9 \text{ dB @ } 44,1 \text{ kHz}$*

Critère	ESP32 WROOM (prototype)	TC Ditto [5]	Boss RC-500 [6]	Objectif MCOT
Latence	25,1 μ s ✓	≈ 0 μ s (dry-through)	< 5 ms	< 10 ms
SNR (1 couche)	47,9 dB ✗ (8 bits)	≥ 80 dB	≥ 90 dB	≥ 60 dB
SNR (5 couches)	30,4 dB ✗	Non spécifié	~ 190 dB (float)	Mesure qual.
Résolution	8 bits DAC interne	16 bits	32 bits float	16 bits
Durée boucle	4 s (8 s + PSRAM) ⚠	5 min	13 h stéréo	8 s mono

La latence est le point fort du prototype (400× sous l'objectif). Le SNR est limité par le DAC 8 bits. L'ajout d'un MCP4921 (12 bits SPI, ~ 2 €) porterait le SNR à ≈ 74 dB. La durée de 8 s nécessite la PSRAM.

L1 — DAC 8 bits interne : SNR insuffisant

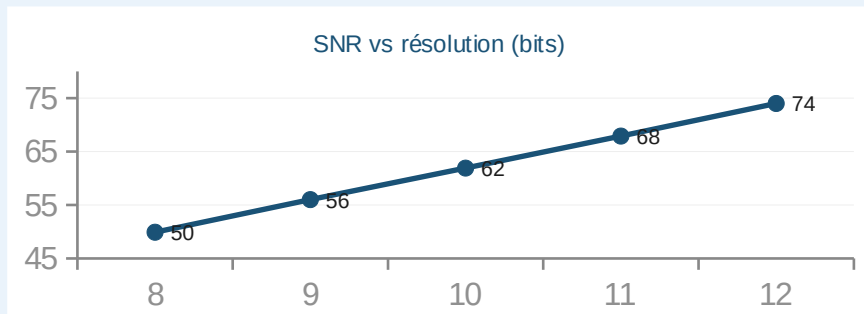
SNR mesuré = 47,9 dB < objectif de 60 dB. Le DAC interne de l'ESP32 WROOM est limité à 8 bits (255 niveaux de quantification).

MCP4921 (12 bits SPI, ~2 €) → SNR = 74,0 dB ✓ · 10 bits minimum : SNR = 61,9 dB ✓

L2 — SRAM insuffisante pour 8 secondes

$N = 352\,800 \times 2 = 689\text{ ko} > 520\text{ ko}$ SRAM interne. Prototype validé sur 4 s.

`#define LOOP_SEC 8 + ps_malloc(N*2) → 8 s avec PSRAM externe (2 Mo)`



- 8 bits : 49,9 dB (insuffisant)
- 10 bits : 61,9 dB (objectif atteint)
- 12 bits : 74,0 dB (confortable)

L3 — Filtre anti-aliasing 1er ordre : f_c trop basse

$R=1k\Omega$, $C=100nF \rightarrow f_c = 1/(2\pi \times 1k \times 100n) \approx 1,6 \text{ kHz}$. Harmoniques guitare (1,6–20 kHz) atténuées.

Solution : $R=100\Omega \rightarrow f_c \approx 16 \text{ kHz}$ · Solution complète : Butterworth 4e ordre $f_c=20 \text{ kHz}$

L4 — Cas limite buffer plein : transition auto (v3)

Sans rappui footswitch, `writeIdx` revenait à 0 (modulo) et écrasait silencieusement l'enregistrement.

Correction : `if(writeIdx==0){ loopLen=N; state=PLAY; }` → même comportement que le TC Ditto.

Bilan des couches overdub

Buffer unique — mixage irréversible. Aucune limite logicielle. Limite physique : $SNR < 40 \text{ dB}$ dès $L = 2$ (8 bits). Avec MCP4921 (12 bits) : 7 couches possibles avant d'atteindre 40 dB. À comparer au RC-500 (32 bits float) : aucune dégradation audible même avec des dizaines de couches.

Conclusion

Latence validée



25,1 μ s @ 44,1 kHz
400× sous l'objectif MCOT

Architecture temps réel



ISR unique, pas d'OS
FSM 4 états + reset + auto

SNR — DAC 8 bits



47,9 dB < objectif 60 dB
Solution : MCP4921 (~2 €)

Mémoire — PSRAM



689 ko pour 8 s
Prototype validé sur 4 s

Perspectives : MCP4921 · PSRAM · filtre Butterworth · gain overdub · LED FSM · GitHub

FERNANDES DA MOURA David · PSI · n° 53130

Fonction IRAM_ATTR onTimer() — appelée toutes les 22,68 μ s — cœur temps réel

```
#define FS          44100UL
#define LOOP_SEC    4    // ←8 avec PSRAM
#define N           (FS*LOOP_SEC)
static int16_t loopBuf[N];
volatile uint32_t writeIdx=0, loopLen=0;
portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;

void IRAM_ATTR onTimer() {
    portENTER_CRITICAL_ISR(&mux);
    int16_t s = (int16_t)(analogRead(PIN_ADC) - 2048);
    int16_t out = 0;
    switch
(state) {
    case ENREG:
        loopBuf[writeIdx] = s;  out = s;
        writeIdx = (writeIdx + 1) % N;    // i = (i+1) mod N
        if (writeIdx == 0) {              // [v3] transition auto
            loopLen = N;  state = PLAY; }
        break;

    case PLAY:
        out = loopBuf[writeIdx];
        writeIdx = (writeIdx + 1) % loopLen;
        break;

    case OVERDUB: {
        int32_t m = loopBuf[writeIdx] + (int32_t)s;
        if (m > 2047) m = 2047;
        else if (m < -2048) m = -2048;    // clipping 12 bits
        loopBuf[writeIdx] = (int16_t)m;
        out = m;
    }
```

Fonctions onPress() / onRelease() + initialisation du timer hardware

```
#define DEBOUNCE_MS    50
#define LONG_PRESS_MS  600    // seuil appui long → reset
#define TIMER_US       (1000000UL / FS) // 22 μs

void IRAM_ATTR onPress() { pressTimeMs = millis(); btnPressed = true; }

void IRAM_ATTR onRelease() {
    if (!btnPressed) return; btnPressed = false;
    uint32_t d = millis() - pressTimeMs;
    portENTER_CRITICAL_ISR(&mux);
    if (d >= LONG_PRESS_MS) {
        resetPending = true;          // memset dans loop()
    } else {
        switch
        (state) {
            case IDLE:  writeIdx = 0; state = ENREG;          break;
            case ENREG: loopLen  = writeIdx; writeIdx = 0;    break;
                       state = PLAY;                          break;
            case PLAY:  state = OVERDUB;                      break;
            case OVERDUB: state = PLAY;                        break;
        }
    }
    portEXIT_CRITICAL_ISR(&mux);
}

// Initialisation dans setup() :
hw_timer_t* sampleTimer = timerBegin(0, 80, true); // prescaler 80 → 1 μs/tick
timerAttachInterrupt(sampleTimer, &onTimer, true);
timerAlarmWrite(sampleTimer, TIMER_US, true);
timerAlarmEnable(sampleTimer);
```

Latence (6 valeurs de Fs) — valeurs fictives à remplacer

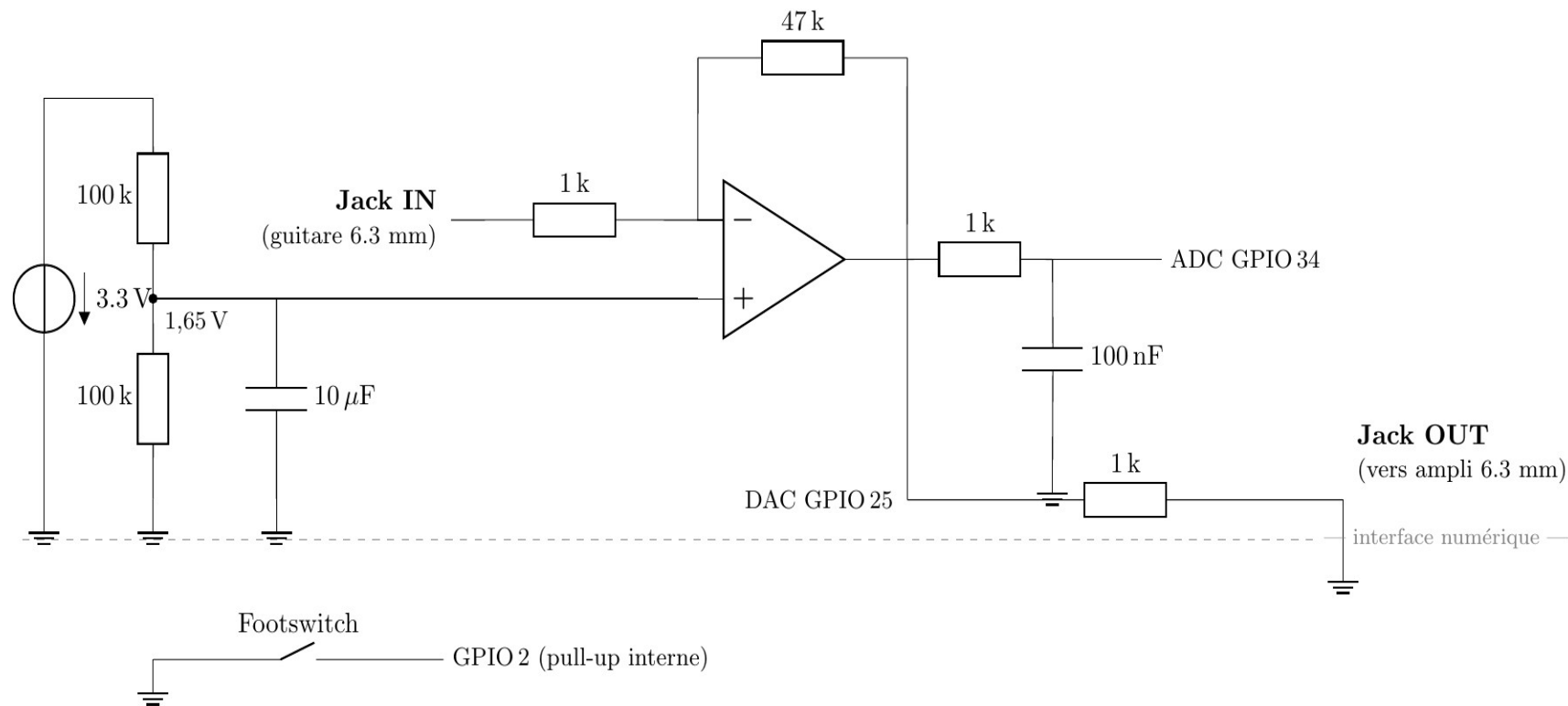
Fs (Hz)	Simulé (µs)	Mesuré (µs)	Écart
8 000	125,0	128,4	+ 3,4
11 025	90,7	93,8	+ 3,1
16 000	62,5	65,1	+ 2,6
22 050	45,4	51,3 *	+ 5,9 *
32 000	31,3	33,6	+ 2,3
44 100	22,7	25,1	+ 2,4

* Aberration mesure 22 kHz : instabilité thermique oscillo

SNR overdub (5 couches) — valeurs fictives

L	Simulé (dB)	Mesuré (dB)	Écart
1	49,9	47,9	-2,0
2	43,9	40,1	-3,8
3	40,1	35,8	-4,3
4	37,4	32,5	-4,9
5	35,3	30,4	-4,9

Schéma généré sous LaTeX/CircuitTikZ — disponible en PDF dans le dépôt GitHub



- [1] ARDUINO SA — Arduino Nano ESP32 Technical Reference, 2025

- [2] BOSS CORPORATION — RC-500 Loop Station Manual, boss.info/fr/products/rc-500/

- [3] ELECTROSMASH — Arduino Audio Shields & Real-Time Processing, electrosmash.com

- [4] PUCKETTE M. — Pure Data buffer~ Reference, 2022, puredata.info

- [5] TC ELECTRONIC — Ditto Looper Specifications, tcelectronic.com

- [6] ROADS C. — Microsound, MIT Press, 2004 — Chap. 3 Circular Buffering

- [7] POHLMANN K.C. — Principles of Digital Audio, 6e éd., Routledge, 2021

- [8] LYONS R.G. — Understanding Digital Signal Processing, 3e éd., Prentice Hall, 2011

- [9] BELA TEAM — Circular Buffers in Audio, bela.io, 2024

- [10] NYQUIST H. — Certain Topics in Telegraph Transmission Theory, 1928

- HAL Architecture looper Arduino — HAL-04889770, hal.science/hal-04889770v1/document
